BeeBot Hackathon Plan: Bee Videos to Micro-Robot Pollinators

Oct 16, 2026 · @ananya

Project summary

BeeBot turns bee-pollination video into structured flight data with Perceptron's perception models, then uses that data to drive a simulated micro-robot pollinator. With 4 people and 20 hours, we build a thin end-to-end slice first and polish second.

The demo (3 minutes): a bee clip plays with live tracking overlays (box, trail, behavior label). Beside it, an analytics panel shows flower visits and dwell time. Beside that, a simulated robot reproduces the bee's flight path to the same flower.

Success criteria

- The pipeline runs end to end on at least 10 clips without manual fixes.
- We report one honest accuracy number: model track vs. ground-truth track on the Monash evaluation videos.
- The sim robot reaches and "pollinates" a flower using extracted trajectories.
- We have a 3-minute live demo plus a recorded backup video.

Assumptions to confirm in hour 0

- Whether the hackathon requires Isaac specifically or accepts Perceptron Mk1.5. Mk1.5 takes video directly; Isaac is image-first, so it needs sampled frames.
- Mk1.5 video limits (length, file size, rate limits, pricing), from its model card.
- Licenses on any dataset or footage we show publicly.

Team roles

Four workstreams, seven steps each (A1–A7, B1–B7, C1–C7, D1–D7), about 2.5 hours per step, so the load is even. Swap in real names for Person A–D.

Workstream	Owner	Mission	Main deliverable
A. Perception	Person A	Get reliable bee and flower tracks out of Perceptron Mk1.5 (and Isaac per frame)	`tracks/*.json` for every clip
B. Data and Eval	Person B	Source and film clips, build ground truth, measure accuracy	`clips/`, `truth/*.json`, eval report
C. Simulation	Person C	Build a micro-robot sim that imitates extracted bee trajectories	`rollouts/*.mp4` and `rollouts/*.json`
D. Demo and Story	Person D	Build the app, overlays, analytics, pitch and backup video	Live demo, deck, backup video

Working rules

- **Contract first.** The schemas in the next section freeze at hour 2. Changes are announced in chat, never made silently.
- **Mock first.** Nobody waits on anyone. C and D build against mock `tracks.json` files until A ships real v1 output at hour 8.
- **Buddy pairs.** A and C pair on `tracks.json` into the sim; B and D pair on eval numbers into the UI. Each pair reviews the other's hardest step.
- **Sync points.** A 10-minute standup at hours 4, 8, 12 and 16: what shipped, what is blocked, what gets cut.
- **Rest.** Stagger 90-minute breaks between hours 9 and 15, with at most one person away at a time. Everyone is awake from hour 16.
- **Stretch goals.** Each person owns one (section 10), started only after their step 5 is done.

Data contract between workstreams

Four file types connect the workstreams. Conventions for all of them: time in seconds, boxes as `[x1, y1, x2, y2]` normalized to 0–1000 with the origin top-left (the same space Mk1.5 returns), and a gap in a track stays a gap. A missing observation is better than an invented one.

1. `clips/manifest.json` (B to everyone)

```
{"clip_id": "monash_001", "path": "clips/monash_001.mp4", "fps": 30,
"duration_s": 12.4,
 "source": "Monash eval video", "license": "verify before public use",
"has_truth": true}
```

2. `tracks/<clip_id>.json` (A to B, C, D)

```
{"clip_id": "monash_001", "model": "perceptron-mk1.5", "prompt_version":
"v3",
 "tracks": [{"track_id": "bee_1", "complete": true,
             "points": [{"t": 0.5, "box": [410, 520, 450, 570]}]}],
 "flowers": [{"flower_id": "flower_1", "box": [560, 300, 700, 440]}],
 "events": [{"track_id": "bee_1", "flower_id": "flower_1", "type": "landing",
             "t_start": 3.2, "t_end": 5.1}]}
```

`events` and behavior labels arrive in step A5; until then the array is empty.

3. `truth/<clip_id>.json` (B to B and D)

Same shape as the tracks file with `"model": "ground_truth"`, built from the Monash tracks or hand-labeled frames.

4. `rollouts/<clip_id>.json` (C to D)

```
{"clip_id": "monash_001", "source_track": "bee_1", "success": true,
 "time_to_flower_s": 4.1, "mean_path_error": 0.03, "video":
"rollouts/monash_001.mp4"}
```

5. `eval/report.json` (B to D)

```
{"clips": 12, "median_center_error": 18, "track_coverage": 0.82,
 "visit_recall": 0.75, "visit_precision": 0.80, "notes": "errors worst under
occlusion"}
```

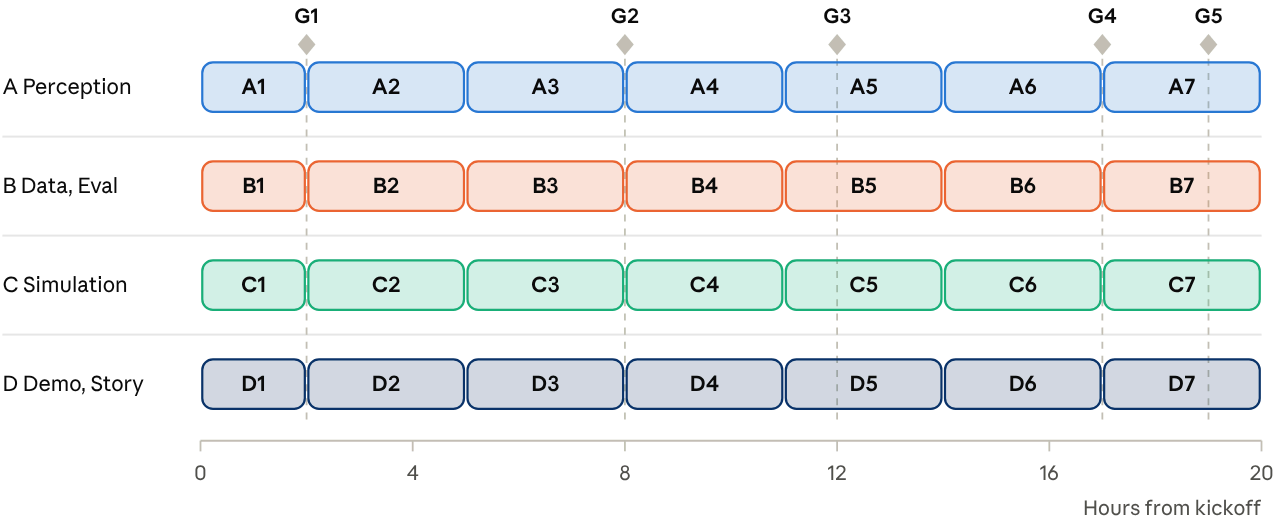
The numbers above are placeholders that show the shape. Real values come from step B4 onward.

20-hour master timeline

Every workstream runs seven steps on the same schedule, so no one finishes early while others are blocked.

All four workstreams ship v1 by hour 8 and meet at hour 12

Steps A1 to D7 on a 20-hour clock; dashed lines are integration gates



G1 contract frozen · G2 tracks v1 · G3 first end-to-end demo · G4 code freeze · G5 submission-ready

20-hour plan · 4 lanes, 7 steps each, 5 gates

Gates G2 (hour 8) and G3 (hour 12) matter most: at G2 real tracks reach the sim and the app, and at G3 the three-panel demo works end to end. Hours 17 to 20 are for freezing, rehearsing and submitting.

Track fit and overall strategy

The Perceptive track rewards creative, impactful use of Perceptron's Mk and Isaac models through the API, out of the box, with no fine-tuning expected. So we spend our effort on application design and a clear demo, not on model training.

What the track asks for. Use image and video capabilities such as object localization, OCR, captioning, tracking and finding events in video, with API access shown at the Perceptron workshop.

How BeeBot maps to it

Track capability	Where we use it	Steps
Object localization	Find bees and flowers in frames	A2, A5
Tracking	Follow a bee through a clip as timestamped boxes	A2–A4
Event finding in video	Locate landings, hovers and flower visits in time	A5, S1

Track capability	Where we use it	Steps
Captioning and visual Q&A	Describe what the bee is doing; search footage in plain language	A5, S1
OCR	Not used. We will not force it into the demo	none

Decisions this track forces

- **No fine-tuning.** We never train a bee detector. The training images and detection models that ship with the Monash datasets stay unused, which is also why we can skip the biggest downloads.
- **Training is allowed downstream.** The simulated robot's controller or policy is our own application code, separate from Perceptron's models.
- **Pitch it as impact.** The story is "turn any pollination video into robot-ready behavior data," which serves pollinator monitoring and robot-pollination research.
- **Attend the Perceptron workshop** and record API credits, rate limits, video limits and which models are allowed. Update the assumptions in the summary the same hour.

The plan in five phases

Phase	Hours	Objective	Exit criteria
1. Foundations	0–2	Everyone can run something	Repo works, schemas frozen, 5 clips with A, API and sim hello worlds started (gate G1)
2. Pipeline v1	2–8	First real data flows	Real `tracks/*.json` for 5 clips load in the sim and the app (G2)
3. Integrate	8–12	One end-to-end path	Pick a clip and see overlay, analytics and sim rollout together (G3)
4. Improve and measure	12–17	Better tracks, defensible numbers, stretch goals	Final accuracy numbers locked and code frozen (G4)
5. Freeze and present	17–20	A demo that cannot fail	Offline demo from a clean checkout, backup video, submission (G5)

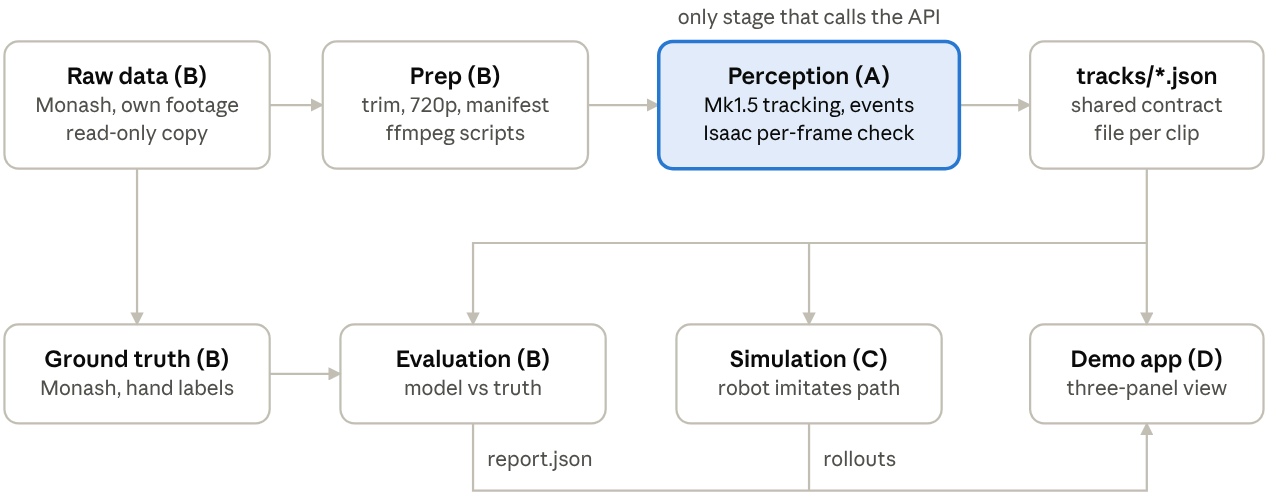
Five working principles

- **Thin slice first.** A rough version of every stage beats a polished version of one.
- **Mock first.** Build against fake files that follow the contract, then swap in real ones.
- **Cache everything.** Save every API response, so reruns are free and the demo works offline.
- **Measure honestly.** One real accuracy number, and one failure case shown on purpose.
- **Demo from cache.** A live API call is a bonus and never the critical path.

System architecture

The whole system is a chain of file-producing stages. Only the Perception stage touches the internet; everything downstream reads `tracks/*.json`, so C and D build on mock files and switch to real ones at hour 8.

Perception is the only API stage; every other part reads its files



system architecture · 8 components, 1 API stage

Ground truth and tracks meet only in Evaluation, so we never grade the model against its own output.

Components

Component	Runs on	Reads	Writes	Owner
Prep scripts	Laptop (`ffmpeg`)	`raw/` , Monash track files	`clips/` , `manifest.json` , `truth/`	B

Component	Runs on	Reads	Writes	Owner
Perception pipeline	Laptop, calling the Perceptron API (Isaac locally or on Colab for the comparison)	`clips/`, `prompts/`	`tracks/`, raw responses in `tracks/raw/`	A
Evaluation	Laptop	`tracks/`, `truth/`	`eval/report.json`	B
Simulation	Laptop (headless render)	`tracks/`	`rollouts/`	C
Demo app	Laptop (Streamlit or Gradio)	`clips/`, `tracks/`, `rollouts/`, `eval/`	nothing (read-only)	D

Repository layout

```
beebot/
  raw/         read-only originals, never in git
  clips/       trimmed 720p working clips + manifest.json
  truth/       ground-truth tracks in contract format
  tracks/      model output per clip; tracks/raw/ keeps every API response
  rollouts/    sim videos and result json
  eval/        run_eval.py, report.json
  sim/         environment, track loader, controller
  app/         overlay renderer, analytics, Streamlit app
  prompts/     versioned prompts (v1, v2, ...)
  scripts/     download, proxy, validate
  CHECKSUMS.txt
```

Git holds code, prompts and small JSON only. `raw/`, `clips/` and `rollouts/*.mp4` are listed in `.gitignore` and shared another way (see the data plan).

Data plan: handling the 19 GB download

Do not work with 19 GB. Download it once on one machine, extract only the evaluation videos and track files, and turn them into about 20 small clips that the whole team shares. This track is out-of-the-box with no fine-tuning, so the large training sets are not needed.

I could not see the file listings on the Monash pages, so I don't know which package is the 19 GB one. Step 1 finds out, and every step below works on any large archive.

Why not use it all

- The API takes one clip at a time, and short windows track better. The docs advise narrowing the interval when answers hit the output limit.
- Local video paths and bytes are sent inline with each request, so big files make every call slow. The SDK can upload a file once and reuse its ID.
- Download time is rough arithmetic that ignores overhead: 19 GB is about 152 Gb, so about 2.1 hours at 20 Mbps, 51 minutes at 50 Mbps and 13 minutes at 200 Mbps. Four people downloading at once would make it worse.
- An archive plus its extracted files can need about twice its size, so plan for around 40 GB free on one machine.

What to keep and what to skip

Item	Decision	Reason
The 10 evaluation videos (Monash spatial-monitoring set)	Keep	API input and comparison with ground truth
Track files with flower visits	Keep	Ground truth for evaluation and targets for the sim
Up to about 100 annotated frames	Keep a subset	Isaac per-frame comparison and a localization check
Our own footage	Keep	Demo material we fully control
Detector training images and models (YOLO)	Skip	No fine-tuning in this track
Analysis software and extra videos	Skip for now	Not on the critical path
Full-size copies on every laptop	Skip	Share small working clips instead

Step 1. Identify what the 19 GB is (B, before hour 0, 15 min)

Open the dataset page's file list and write down every file name, size and format. Note whether it is one big zip or separate files, and read the license line. Save this as `DATA_NOTES.md`. Done when you can name the 10–20 files you need.

Step 2. Free up disk (B, 5 min)

Run `df -h .` and make sure you have at least twice the archive size plus 5 GB free. If not, use an external SSD.

Step 3. Start the download early on one machine (B, before hour 0)

Use a resumable tool, a wired or strong connection, and keep the laptop awake (`caffeinate -i` on macOS). If the data comes as separate files, download the evaluation videos and track files first.

```
mkdir -p beebot/raw && cd beebot/raw
aria2c -c -x 8 -s 8 "<file-url>"      # preferred: parallel and resumable
# or: wget -c "<file-url>"
# or: curl -L -C - -O "<file-url>"
```

Step 4. Verify the download (B, 10 min)

Compare the file size with the listing. If a checksum is published, check it with `shasum -a 256 <file>` (or `md5sum`). For a zip, `unzip -l <file>.zip | head` confirms it opens; `unzip -t` tests the whole archive but takes a while.

Step 5. Extract only what you need (B, 20 min)

List the archive, then extract selected folders. The pattern below is a placeholder; use the real folder names from the listing.

```
unzip -l archive.zip | sort -k1 -n -r | head -30      # find the biggest
files
unzip archive.zip "*evaluation*/*" -d raw/            # extract selected
paths only
```

Step 6. Lock the originals (B, 2 min)

Run `chmod -R a-w raw/` and confirm `.gitignore` lists `raw/`, `clips/` and `rollouts/*.mp4`.

Step 7. Inspect the videos (B, 15 min)

```
find raw -name "*.mp4" -exec ffprobe -v error -select_streams v:0 \
  -show_entries format=filename,duration:stream=width,height,r_frame_rate \
  -of default=nw=1 {} \;
```

Note resolution, frame rate and duration for each video in `DATA_NOTES.md`.

Step 8. Choose clip windows (B, 30 min)

Use the Monash track files to find times when a flower visit happens. Pick 5–15 s windows: about 10 with a visit, about 5 with partial occlusion and about 5 quiet ones, then add our own footage. Record them in `windows.csv` with columns `clip_id,source,start_s,duration_s`.

Step 9. Cut small working clips (B, 30 min)

```
tail -n +2 windows.csv | while IFS=, read id src start dur; do
  ffmpeg -y -ss "$start" -i "$src" -t "$dur" -vf "scale=-2:720" \
    -c:v libx264 -crf 23 -preset fast -an "clips/$id.mp4"
done
```

Twenty 10-second clips at 720p should land far below 1 GB (rough estimate). Bees in wide shots may vanish at 720p, so A tests one wide shot at native resolution in step A2. If native is needed, keep both versions and note the resolution in the manifest.

Step 10. Write the manifest (B, 20 min)

Generate `clips/manifest.json` from `ffprobe` output with the fields from the data contract (`clip_id` , `path` , `fps` , `duration_s` , `source` , `license` , `has_truth`). A later step adds `file_id`.

Step 11. Convert ground truth (B, with step B3)

Convert Monash tracks to `truth/<clip>.json`. Normalize pixels with x / width × 1000 and y / height × 1000, convert frame numbers to seconds with frame / fps, subtract the window start, and keep gaps as gaps. Done when D's overlay of the truth lines up with the video on 3 clips.

Step 12. Share the small set, don't re-download (B and D, 20 min)

```
shasum -a 256 clips/* truth/* > CHECKSUMS.txt
cd beebot && python -m http.server 8000      # or use a shared drive or USB
stick
```

Teammates pull from `http://<your-ip>:8000/clips/` (or the shared drive), then run `shasum -a 256 -c CHECKSUMS.txt` to confirm their copy matches.

Step 13. Upload once to the API and cache the IDs (A, during A4)

Upload each clip once with the SDK's Files feature, store the returned ID as `file_id` in the manifest, and reuse it across prompts. Read the Mk1.5 model card and Files guide first for limits; we have not verified them.

Step 14. Back up (B, 10 min)

Copy `clips/`, `truth/` and `tracks/` to a second drive. Before the demo, copy the demo set to the presentation laptop.

If the download is slow or fails

1. Take only the 10 evaluation videos and track files.
2. Switch to the Monash honeybee video tracking data, or the CC0 hawkmoth set.
3. Lean on our own footage.

Any of these still gives 10 or more clips, which is enough for the full plan. Do not block the team on the download: until it finishes, A and C work with own footage and any public clip.

Workstream A: Perception (Person A)

A turns video into `tracks/*.json`. The first real output ships at hour 8, and everything after that makes it more reliable.

Step	Hours	Goal	Input	Output	Done when
A1. Setup and hello world	0–2	One working API call	API key from platform.perceptron.inc; `pip install "perceptron>=0.4.0"`; one bee clip	`hello_track.py`	A `<track>` comes back and parses with `response.annotations(strict=True)`
A2. Prompt tuning	2–5	A prompt that reliably tracks one bee	5 varied clips from B; prompt template	`prompts/v1-v3.txt` and a notes file on what worked	Track looks right in an overlay on at least 4 of 5 clips
A3. Parser to schema	5–8	Contract-valid output	Raw Mk1.5 responses	`tracks/<clip>.json` for the 5 clips, raw response saved beside each	Validator passes (coords in range, boxes ordered, times within duration, finish reason is `stop`); C and D can load it
A4. Batch run and cache	8–11	Every clip processed, repeatably	`clips/manifest.json`	Tracks for all clips; cache keyed by clip + prompt version; cost and latency log	Full run finishes unattended; a rerun makes no API calls
A5. Flowers and events	11–14	Landing, hover and leave events	Tracks; a reference frame per clip	`flowers` and `events` fields filled	Events exist for at least 8 clips and B can compare them with truth visits
A6. Reliability and Isaac comparison	14–17	Fewer failures; a fair model comparison	Failure list from B5	`tracks_v2/`; Mk1.5 vs Isaac per-frame comparison on 3 clips	Failures fixed or documented; truncated answers handled by shorter windows
A7. Freeze and support	17–20	Nothing changes under the demo	Final prompts and cache	Tagged `tracks/` release; pipeline README	The demo runs from cache with the network off

Tips for A

- Describe the target by appearance and starting location, and ask for one bee per request.
- Tell the model not to invent positions when the bee is occluded or off-screen.
- If the finish reason is `length`, shorten the clip window or track fewer objects. Never patch truncated markup by adding closing tags.
- Model tracks can be sparse. Interpolate short gaps client-side and mark them `interpolated` so evaluation stays honest.
- Isaac is image-first, so for the comparison in A6 sample frames at 5–10 fps and call its `detect` or `point` operation.

Workstream B: Data and Eval (Person B)

B supplies the clips and the ground truth, and turns "it looks good" into a number we can defend.

Step	Hours	Goal	Input	Output	Done when
B1. Pick datasets and check licenses	0–2	Usable clips, clear rights	Monash spatial-monitoring and honeybee-tracking datasets; flower-patch assay paper; hawkmoth CC0 set	`clips/` raw downloads, `LICENSES.md`, `manifest.json` v0	At least 5 varied clips are handed to A by hour 2
B2. Film and trim own footage	2–5	Demo-quality clips we own	Phone camera, flowers with pollinators, `ffmpeg`	10 or more trimmed 5–15 s clips (steady, close, 30 fps or slow-mo)	Manifest lists at least 20 clips total, 10 or more usable
B3. Build ground truth	5–8	Truth in contract format	Monash track files; 10 hand-labeled frames on each of 5 own clips (CVAT or Label Studio)	`truth/<clip>.json`	Truth exists for at least 8 clips and the loader test passes
B4. Metric script v1	8–11	A repeatable scoring tool	A's first `tracks/*.json` plus truth	`eval/run_eval.py`; `eval/report.json` v1	One command scores the first 5 clips

Step	Hours	Goal	Input	Output	Done when
B5. Full eval and error analysis	11–14	Know where it fails	Tracks for all clips	Report on all clips; FiftyOne view of failures; failure categories	Failure list sent to A by hour 13
B6. Close the loop and pick demo clips	14–17	Final numbers and story clips	A's `tracks_v2/`; failure categories	`eval/report_v2.json`; 3 demo clips (clean win, tricky case, honest failure); 2 charts	Final numbers locked by hour 17
B7. Fact-check, QA, submission	17–20	No wrong claims on stage	Deck, app, report	Attribution slide; checklist of verified claims; submission package	Every number on a slide matches `report_v2.json`; the demo runs from a clean checkout

Metrics to compute

- **Median center error:** distance between model and truth box centers, in normalized units.
- **Track coverage:** share of truth frames that have a model observation.
- **Visit recall and precision:** flower-visit events matched within a 0.5 s window.

Tips for B

- Trim with `ffmpeg -ss 00:00:03 -i in.mp4 -t 10 -c copy out.mp4`.
- Outdoor pollinator activity drops in autumn depending on your region. Scout a flowering patch early in B2, and if there is none, lean on the datasets and treat own footage as a bonus.
- Keep a one-line provenance note per clip so the attribution slide takes minutes, not an hour.

Workstream C: Simulation (Person C)

C builds a simulated micro-robot that imitates the bee trajectories A extracts. Until hour 8, C works on mock tracks that follow the contract.

Step	Hours	Goal	Input	Output	Done when
C1. Pick and install the sim	0–2	A sim that renders video headless	MuJoCo or PyBullet (use whichever someone knows); fallback: 2D top-down in pygame or matplotlib	`sim/hello.py`	It writes an mp4 on a machine without a display
C2. Environment v1	2–5	Flower target and a robot body	Sim of choice	`sim/env.py` with reset, step and render	A random policy runs 1,000 steps and renders
C3. Trajectory loader and mapping	5–8	Bee path in sim coordinates	Mock `tracks.json`, then A's real files at hour 8	`sim/load_tracks.py`; plot of the mapped path	Real track files load with no code changes
C4. Imitation controller	8–11	Robot follows the bee's path	Mapped trajectory	`sim/follow.py` (waypoint follower with PD control and speed limits); rollouts for 5 clips	Robot gets within the flower radius on at least 3 of 5 clips
C5. Pollination logic	11–14	A clear, measurable success	A's `events` (landing and dwell times)	Success rule (within radius for a dwell time), pollen counter, `rollouts/*.json`	Success flag and path error logged for every clip
C6. Render polish	14–17	Videos ready for the app	Rollouts	`rollouts/*.mp4` matched to clip duration and fps, with trail and counter overlay	D can drop them into the three-panel view
C7. Freeze, backup, talk track	17–20	A robust, honest story	Final rollouts	Backup copies; one paragraph on how the mapping works and what it leaves out	Demo plays with the sim offline

Design decisions to make in C1–C3

- **2D vs 3D.** One camera gives only image-plane motion. Map the path onto a plane at a fixed depth and say so plainly. Depth is a stretch goal.
- **Time scaling.** A real micro-robot cannot match bee speed. Slow the path by a constant factor and show the factor on screen.

- **Scope of the claim.** We imitate behavior (path, hover, landing), not wing physics. Present it as a behavioral prototype that a real flapping-wing controller could later track.

Workstream D: Demo and Story (Person D)

D owns the repo, the app and the pitch. D builds against mock data first, so the UI is ready the moment real tracks arrive.

Step	Hours	Goal	Input	Output	Done when
D1. Repo, board and app skeleton	0–2	Everyone can run the project	Folder layout from the contract; Streamlit or Gradio	Git repo with `clips/ tracks/ truth/ rollouts/ eval/ sim/ app/`; README; task board; mock data files	All four teammates clone and run the hello app
D2. Overlay renderer	2–5	Tracks drawn on video	Mock tracks and a clip	`app/overlay.py` writing an annotated mp4 (box, trail, behavior label)	An overlay plays smoothly on one clip
D3. Analytics panel	5–8	Numbers a judge can read in 5 seconds	Mock tracks and events	`app/analytics.py`: visits per flower, dwell time, flight heatmap	Charts render from mock data
D4. Three-panel app on real data	8–11	The demo skeleton	A's tracks, C's rollouts, B's report	Streamlit view: annotated video, analytics and sim side by side, with a clip selector	Picking a clip updates all three panels from real files
D5. Story and deck	11–14	A clear 3-minute narrative	Results so far	Deck v1 (problem, approach, results, vision), script and speaker split	Team has read and commented on the deck
D6. Polish and backup video	14–17	A demo that cannot fail	Working app	Loading states, cached mode with no network, recorded 3-minute backup video	Backup video exists and the app runs offline
D7. Rehearse and submit	17–20	Confident delivery	Final deck and app	Three timed rehearsals; Q&A cheat sheet; submission	Last rehearsal fits in 3 minutes with 30 seconds spare

Tips for D

- Precompute overlays and charts offline and play cached files in the app. Call the live API only in an optional "analyze a new clip" button.

- Use only facts we can cite in the deck. Cite sources for any claim about pollinators or robot pollination.
- Show one honest failure case. Judges trust a team that knows where the model breaks.

Integration milestones and demo script

Five gates keep four parallel workstreams from colliding. If a gate is missed by more than an hour, use the cut list in section 11.

Gate	Hour	What must be true	Who checks
G1. Contract and clips	2	Schemas frozen; 5 clips in A's hands; mock data in C's and D's hands	D
G2. Tracks v1	8	Real `tracks/*.json` for 5 clips load in the sim and the app	A and C
G3. First end-to-end demo	12	Pick a clip and see overlay, analytics and sim rollout together	Whole team
G4. Code freeze	17	Final numbers locked; only bug fixes after this	B and D
G5. Submission-ready	19	Demo runs offline from a clean checkout; backup video recorded	B

Hour 20 is buffer, not plan. Submit as soon as G5 is met.

Three-minute demo script

Time	Beat	Speaker	On screen
0:00–0:20	Hook: robot pollinators need to learn from real pollinators	D	Title slide, bee clip
0:20–0:50	Approach: video in, Perceptron tracks out, robot imitates	A	One-slide pipeline
0:50–1:30	Live: choose a clip, see tracking overlay and flower-visit analytics	A and D	App, panels 1 and 2

Time	Beat	Speaker	On screen
1:30–2:10	The sim robot follows the same path to the same flower	C	App, panel 3 beside the bee
2:10–2:40	Results: accuracy vs ground truth, plus one honest failure	B	Eval chart, failure clip
2:40–3:00	Vision: from sim to a real micro-robot; what we would build next	D	Closing slide

Demo-day rules

- Play cached outputs by default. The live API call is a bonus, never the critical path.
- Keep the backup video one click away and tested on the presentation machine.
- Assign one person to watch the clock and one to handle questions.

Stretch goals and plans

Five stretch goals, each with an owner and a hard time-box. Start one only when its owner's step 5 is done and gate G3 (hour 12) is met. Stop all stretch work at hour 17.

Stretch goal	Owner	Starts after	Time-box	Why it wins points
S1. Ask your footage anything	A	A5	3 h	Natural-language search over video is easy to grasp and fun to try live
S2. Live "analyze a new clip" mode	D	D4	2.5 h	Judges can hand us their own clip
S3. Learn, don't just copy	C	C5	3 h	Moves the story from imitation to a trained policy
S4. Who pollinates best?	B	B5	3 h	Adds a scientific finding beyond tracking
S5. Edge run with Isaac	A or C	A6	2 h	Shows the "small model on a robot" story

S1. Ask your footage anything (A, with D for the UI)

1. Test Mk1.5 video understanding on 3 clips with prompts like "When does a bee land on a flower? Give timestamps."
2. Parse the returned time ranges and validate them against clip duration.
3. D adds a search box that jumps the video to each result.
4. Done when 5 sample queries return sensible clips on the demo videos.

S2. Live analyze mode (D)

1. Add an upload control and a "track" button that calls the same A-pipeline function.
2. Show progress and a 60-second timeout.
3. On timeout or error, fall back to the nearest cached clip with a clear label.
4. Done when a new 10-second clip goes from upload to overlay without a code change.

S3. Learn, don't just copy (C)

1. Wrap the env as a Gymnasium environment with the success rule as the reward.
2. Train with Stable-Baselines3 (PPO); optionally warm-start with behavior cloning on bee paths.
3. Compare success rate and path error against the pure follower from C4.
4. Done when we can show one chart: imitation vs trained policy.

S4. Who pollinates best? (B)

1. Use the Monash tracks, which cover four insect varieties: honeybees, hoverflies (Syrphidae), butterflies and moths (Lepidoptera), and wasps (Vespids).
2. Compute visits per track and dwell time by insect type from the truth files.
3. Test Isaac `classify` on cropped insect frames and report a confusion matrix.
4. Done when one chart compares insect types.

S5. Edge run with Isaac (A or C)

1. Load an open-weights Isaac 0.2 model from Hugging Face (research license is non-commercial) on a laptop or Colab GPU.
2. Run `detect` on sampled frames from one clip and log frames per second.
3. Done when we can say, with a measured number, how fast a small model runs on our hardware.

Priority order if time is short: S1, S2, S3, S4, S5.

Risks, fallbacks and the cut list

The biggest risk is that bees are small and fast, so tracking is the thing most likely to disappoint. Every risk below has a fallback decided in advance.

Risk	Early warning	Fallback
Tracks are sparse or drift on small, fast bees	A2 overlay misses on 2 or more of 5 clips	Crop to the flower region; use shorter windows; one bee per request; interpolate short gaps and label them
Mk1.5 video limits or rate limits block batch runs	Errors or slow responses in A1	Read the model card in hour 0; cache everything; use smaller clips; fall back to Isaac on sampled frames
Hackathon requires Isaac specifically	Organizer answer in hour 0	Keep Isaac per-frame detection as the main path for A and use Mk1.5 only for the comparison and S1
Dataset license blocks public use	B1 license check	Use own footage and CC0 data in the public demo; keep restricted data for private evaluation
Sim is slow to set up or won't render headless	C1 not finished by hour 3	Switch to a 2D top-down sim in pygame or matplotlib
Live demo fails	Any network hiccup in rehearsal	Cached mode by default; backup video ready
Integration collision at hour 8 or 12	Mock and real files disagree	Validator script in CI or pre-commit; contract changes announced in chat
Burnout	Mistakes rising after hour 12	Staggered 90-minute breaks; cut scope before cutting sleep

Cut list (drop in this order if behind)

1. All stretch goals.
2. Flight heatmap and behavior labels (keep boxes and trails).

3. Isaac comparison in A6.
4. Own footage (use public datasets only).
5. Trained policy in the sim (keep the waypoint follower).

Never cut: the end-to-end path, the one accuracy number, the backup video.

Tools, accounts and pre-flight checklist

Area	Tool	Used by
Perception API	Perceptron Python SDK (`pip install "perceptron>=0.4.0"`), `PERCEPTRON_API_KEY`	A, S1, S2
Isaac open weights	Hugging Face `PerceptronAI/Isaac-0.2-2B-Preview` (CC-BY-NC 4.0)	A6, S5
Video and frames	`ffmpeg` , OpenCV	A, B, D
Labeling and inspection	CVAT or Label Studio; FiftyOne	B
Simulation	MuJoCo or PyBullet; Gymnasium and Stable-Baselines3 for S3	C
App and charts	Streamlit or Gradio; matplotlib or Plotly	D
Collaboration	One git repo, shared task board, one team chat	All

Pre-flight checklist (do before hour 0 if possible)

- [ ] Everyone has a Perceptron platform account and API key; one shared test call works.
- [ ] Ask organizers: is Isaac required, or is Mk1.5 accepted? Are there API credits?
- [ ] Read the Mk1.5 model card for video limits and pricing.
- [ ] Repo created and all four teammates can push.
- [ ] Python 3.10 or newer and `ffmpeg` installed on every laptop.
- [ ] Datasets downloading (Monash files can be large) on the venue network.
- [ ] One teammate checks a nearby flowering patch for own footage.
- [ ] Names written into the roles table.

Sources

- Perceptron Mk1.5 quickstart

- Perceptron Mk1.5 video tracking
- Perceptron Python SDK
- Isaac 0.2 2B Preview model card
- Isaac-0.1 FiftyOne integration
- Monash Spatial Monitoring and Insect Behavioural Analysis Dataset
- Monash Honeybee video tracking data
- Towards Automatic Honey Bee Flower-Patch Assays (arXiv)
- UW hawkmoth foraging dataset (CC0)